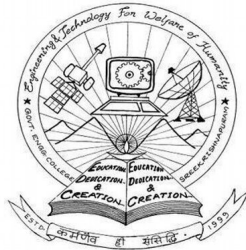


ITT464 COMPUTER GRAPHICS

Module 1: Graphics Systems



Anoop S K M

(<https://sites.google.com/site/skmanoop>)

Department of Information Technology

Govt. Engg. College, Sreekrishnapuram, Palakkad

Acknowledgements

- All the pictures are taken from the Internet using Google search.
- Wikipedia also referred.
- Some pictures are from Computer Graphics by Hearn and Baker

Meeting 00

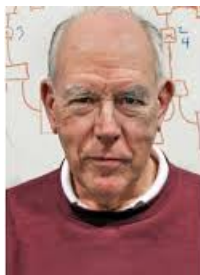
Father of Computer Graphics



"It's not the school, the curriculum, or the teacher, but motivation that is the single most important ingredient in learning."

IVAN SUTHERLAND

SPRINGER NATURE GROUP #OnThisDay

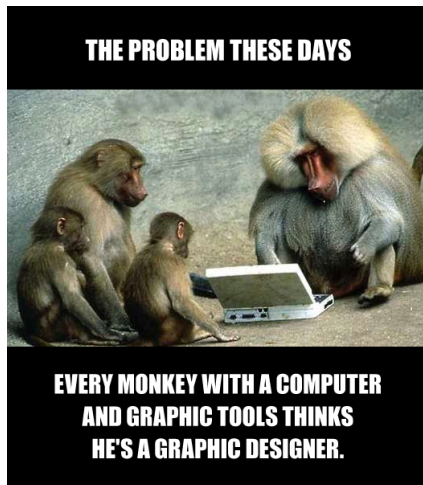


Ivan Edward Sutherland is the Father of Computer Graphics.

He was awarded the Turing Award in 1988 for Sketchpad, World's first Graphical User Interface

- OpenGenus

Welcome to Computer Graphics!



Let's Explore and Understand them in Detail!

What's in Store?

Syllabus

Module 1: Graphics Systems (8 Hours)
Raster Scan & Random Scan systems. Output Primitives – Line Drawing Algorithms (DDA, Bresenham), Bresenham's circle generation algorithm. Filled Area Primitives – Scan Fill, Flood Fill, Boundary Fill. Inside outside tests.
Module 2: Display Technologies(6 Hours)
Working principle behind CRT, LCD, Plasma, LED, OLED, AMOLED, E-Paper displays.
Module 3: 2-Dimensional Geometric Transformations (8 Hours)
Basic Transformations, Reflection & Shear, Homogenous Matrix representation of transformations, Composite Transformations.
Module 4: 2-D Clipping & 3-Dimensional Geometric Transformations (6 Hours)
2-D Clipping: Point Clipping, Cohen-Sutherland Line Clipping Algorithm, Sutherland-Hodgeman Polygon Clipping Algorithm. 3-Dimensional Geometric Transformations: Basic transformations, Composite 3D transformations (Basics only)
Module 5: Visible Surface Detection Methods & Colour Models (7 Hours)
Visible Surface Detection Methods: Back Face Detection, Depth Buffer, A-Buffer, Scan line, Depth sorting methods. Colour Models: RGB Model, CMY Model, HSV Model

Text Books

1. Donald Hearn, Pauline Baker, Computer Graphics – C Version, Pearson Education, 2nd Edition, 2002
2. Amarendra N' Sinha and Arun D Udai, "Computer Graphics", The McGraw-Hill

Meeting 01

Applications

- Computer Aided Design

Applications

- Computer Aided Design
- Presentation Graphics (Bar chart, Pie chart etc.)

Applications

- Computer Aided Design
- Presentation Graphics (Bar chart, Pie chart etc.)
- Computer Art

Applications

- Computer Aided Design
- Presentation Graphics (Bar chart, Pie chart etc.)
- Computer Art
- Entertainment

Applications

- Computer Aided Design
- Presentation Graphics (Bar chart, Pie chart etc.)
- Computer Art
- Entertainment
- Education and Training

Applications

- Computer Aided Design
- Presentation Graphics (Bar chart, Pie chart etc.)
- Computer Art
- Entertainment
- Education and Training
- Visualisation
 - visualising atoms/molecules etc. which are very small

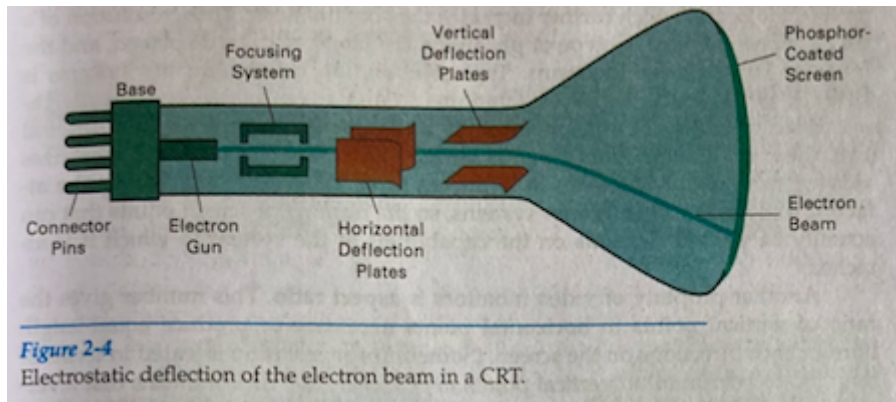
Applications

- Computer Aided Design
- Presentation Graphics (Bar chart, Pie chart etc.)
- Computer Art
- Entertainment
- Education and Training
- Visualisation
 - visualising atoms/molecules etc. which are very small
 - visualising galaxy, milkyway etc. which are very huge
- Image Processing

Applications

- Computer Aided Design
- Presentation Graphics (Bar chart, Pie chart etc.)
- Computer Art
- Entertainment
- Education and Training
- Visualisation
 - visualising atoms/molecules etc. which are very small
 - visualising galaxy, milkyway etc. which are very huge
- Image Processing

Cathode Ray Tube (CRT)



Raster Scan I

Persistence

Time it takes for the emitted light from the screen to decay to one-tenth of its original intensity.

Raster Scan II

Refresh Rate

The number of times a pixel is redrawn in one second.

Resolution

The maximum number of points that can be displayed on a screen without overlapping.

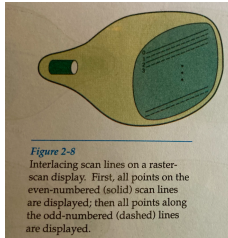
Horizontal Retrace

The return to the left of the screen, after refreshing each scan line

Vertical Retrace

The return of to the top left corner of the screen at the end of each frame

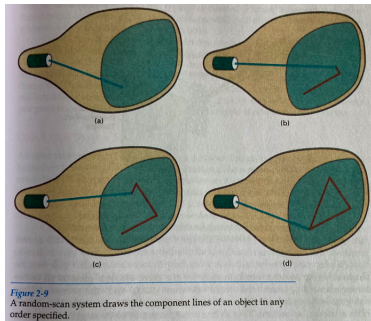
Raster Scan III



Interlaced Refresh

A single frame is displayed in two passes, one in which all the even numbered lines and the other in which all the odd numbered lines are displayed.

Random Scan



- Display file, consisting of vectors having same intensity values

Meeting 02

We Saw & Will See

Till Now We Saw...

- Module 1 : Graphic Systems
 - Raster Scan & Random Scan

Today We Will See...

- Line Drawing Algorithm : DDA

Line Drawing Algorithms

$$y = mx + b$$

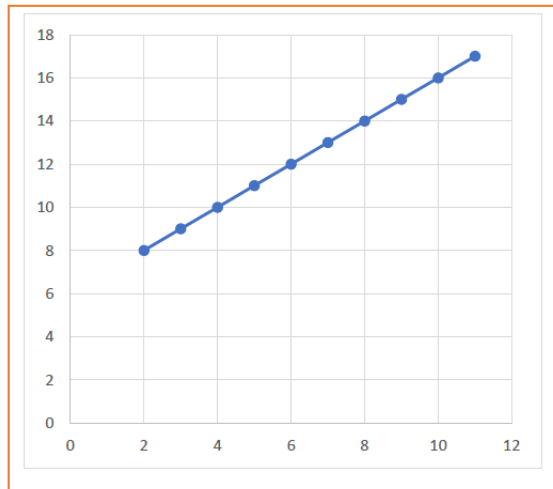
$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

$$b = y_1 - mx_1$$

$$\Delta y = m \Delta x$$

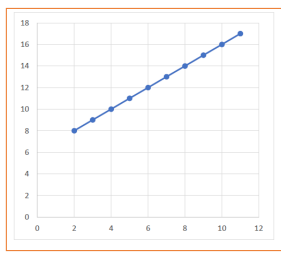
$$\Delta x = \frac{\Delta y}{m}$$

Line Drawing Example



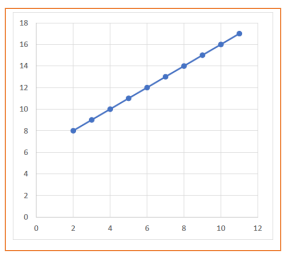
img courtesy: gatevidyalay.com

Digital Differential Analyzer(DDA)



- m is the slope
- $y_{k+1} = y_k + m$ if slope ≤ 1 , sample at unit x intervals

Digital Differential Analyzer(DDA)



- m is the slope
- $y_{k+1} = y_k + m$ if slope ≤ 1 , sample at unit x intervals
- $x_{k+1} = x_k + \frac{1}{m}$ if slope > 1 , sample at unit y intervals

Meeting 03

We Saw & Will See

Till Now We Saw...

- Module 1 : Graphic Systems
 - Raster Scan & Random Scan
 - Line Drawing Algorithm : DDA

Today We Will See...

- DDA Algorithm examples

DDA Algorithm

```

void dda(int x_1, int y_1, int x_2, int y_2)
{
    int dx = x_2 - x_1;
    int dy = y_2 - y_1;
    int steps = abs(dx)>abs(dy)? abs(dx):abs(dy);
    float x_incr = dx/steps;
    float y_incr = dy/steps;

    int x=x_1;
    int y=y_1;
    for(int i=0;i<steps;i++){
        putpixel(x,y);
        x+=x_incr;
        y+=y_incr;
    }
}

```

DDA Algorithm Example

DDA Example

- Suppose we want to draw a line starting at pixel (2,3) and ending at pixel (12,8).
- What are the values of the variables x and y at each timestep?
- What are the pixels colored, according to the DDA algorithm?

$\text{numsteps} = 12 - 2 = 10$
 $\text{xinc} = 10/10 = 1.0$
 $\text{yinc} = 5/10 = 0.5$

t	x	y	R(x)	R(y)
0	2	3	2	3
1	3	3.5	3	4
2	4	4	4	4
3	5	4.5	5	5
4	6	5	6	5
5	7	5.5	7	6
6	8	6	8	6
7	9	6.5	9	7
8	10	7	10	7
9	11	7.5	11	8
10	12	8	12	8

Tutorial 01

Find the intermediate points to be plotted for a line from $(18, 22)$ to $(9, 15)$ using DDA algorithm

Meeting 04

We Saw & Will See

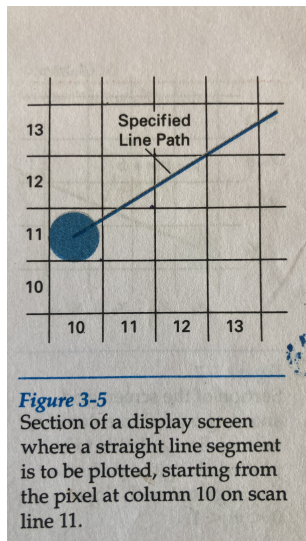
Till Now We Saw...

- Module 1 : Graphic Systems
 - Raster Scan & Random Scan
 - Line Drawing Algorithm : DDA

Today We Will See...

- Bresenham's Line Drawing Algorithm

Bresenham's Line Drawing Algorithm



Bresenham's Line Drawing Algorithm

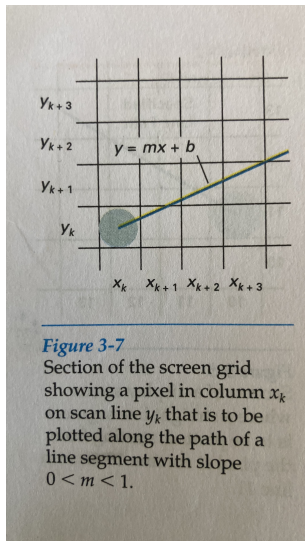
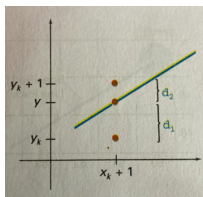


Figure 3-7

Section of the screen grid showing a pixel in column x_k on scan line y_k that is to be plotted along the path of a line segment with slope $0 < m < 1$.

Derivation of Bresenham's I



$$y = m(x_k + 1) + b$$

$$d_1 = y - y_k$$

$$= m(x_k + 1) + b - y_k$$

$$d_2 = y_k + 1 - y$$

$$= y_k + 1 - m(x_k + 1) - b$$

$$d_1 - d_2 = 2m(x_k + 1) - 2y_k + 2b - 1$$

Derivation of Bresenham's II

Let $p_k = \Delta x(d_1 - d_2)$ be the decision parameter.

$$\begin{aligned} p_k &= \Delta x(d_1 - d_2) \\ &= 2\Delta y x_k - 2\Delta x y_k + c \end{aligned}$$

$$p_{k+1} = 2\Delta y x_{k+1} - 2\Delta x y_{k+1} + c$$

$$p_{k+1} - p_k = 2\Delta y(x_{k+1} - x_k) - 2\Delta x(y_{k+1} - y_k)$$

The value of p_{k+1} depends upon whether p_k is positive or negative.

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x, \text{ if } p_k \text{ is positive \& plot } (x_k + 1, y_k + 1)$$

$$p_{k+1} = p_k + 2\Delta y, \text{ if } p_k \text{ is negative \& plot } (x_k + 1, y_k)$$

Bresenham's Line Drawing Algorithm

Bresenham's Line-Drawing Algorithm for $|m| < 1$

1. Input the two line endpoints and store the left endpoint in (x_0, y_0) .
2. Load (x_0, y_0) into the frame buffer; that is, plot the first point.
3. Calculate constants Δx , Δy , $2\Delta y$, and $2\Delta y - 2\Delta x$, and obtain the starting value for the decision parameter as

$$p_0 = 2\Delta y - \Delta x$$

4. At each x_k along the line, starting at $k = 0$, perform the following test:
If $p_k < 0$, the next point to plot is $(x_k + 1, y_k)$ and

$$p_{k+1} = p_k + 2\Delta y$$

Otherwise, the next point to plot is $(x_k + 1, y_k + 1)$ and

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x$$

5. Repeat step 4 Δx times.

Meeting 05

We Saw & Will See

Till Now We Saw...

- Module 1 : Graphic Systems
 - Raster Scan & Random Scan
 - Line Drawing Algorithm : DDA, Bresenham's

Today We Will See...

- Bresenham's Circle Drawing Algorithm (Midpoint Circle)

Circle Equation

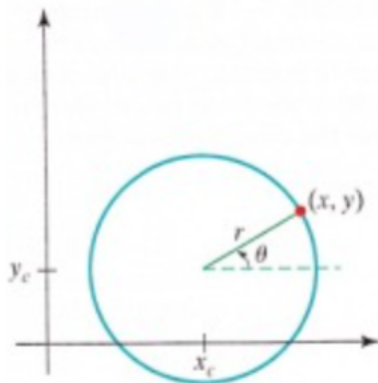


FIGURE 3-16 Circle with center coordinates (x_c, y_c) and radius r .

Circle Drawing

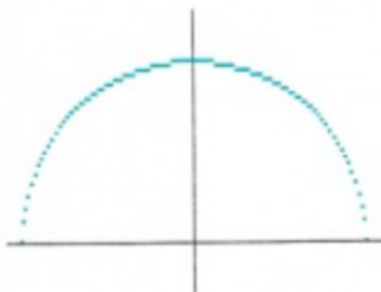


FIGURE 3-17 Upper half of a circle plotted with Eq. 3-27 and with $(x_c, y_c) = (0, 0)$.

Circle Symmetry

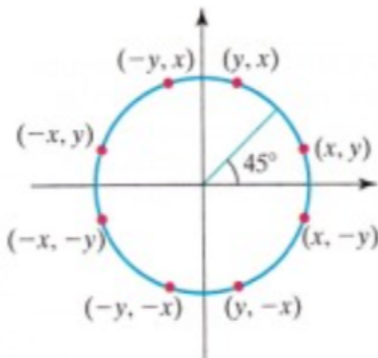


FIGURE 3-18 Symmetry of a circle. Calculation of a circle point (x, y) in one octant yields the circle points shown for the other seven octants.

Bresenham's OR Midpoint Circle Algorithm Concept

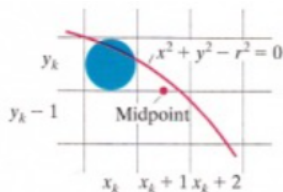


FIGURE 3-19 Midpoint between candidate pixels at sampling position $x_k + 1$ along a circular path.

- If midpoint is inside then select $(x_k + 1, y_k)$ else $(x_k + 1, y_k - 1)$
- Similar to $d_1 - d_2$ in Bresenham's Line drawing we have $f_{circle}(x, y)$
- $f_{circle}(x, y) = x^2 + y^2 - r^2$

Circle Equation

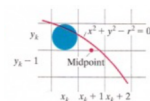


FIGURE 3-19 Midpoint between candidate pixels at sampling position $x_k + 1$ along a circular path.

$$f_{circle}(x, y) = \begin{cases} < 0, & \text{if } (x, y) \text{ is inside the circle boundary} \\ = 0, & \text{if } (x, y) \text{ is on the circle boundary} \\ > 0, & \text{if } (x, y) \text{ is outside the circle boundary} \end{cases}$$

Let $p_k = f_{circle}(x, y)$, where (x, y) is the midpoint of the next two possible pixel candidates.

Finding the parameter p_k I

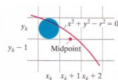


FIGURE 3-19 Midpoint between candidate pixels at sampling position $x_k + 1$ along a circular path.

$$p_k = f_{circle}(x_k + 1, y_k - \frac{1}{2})$$

$$= (x_k + 1)^2 + (y_k - \frac{1}{2})^2 - r^2$$

$$p_{k+1} = f_{circle}(x_{k+1} + 1, y_{k+1} - \frac{1}{2})$$

$$= [(x_k + 1) + 1]^2 + (y_{k+1} - \frac{1}{2})^2 - r^2$$

$$p_{k+1} = p_k + 2(x_k + 1) + (y_{k+1}^2 - y_k^2) - (y_{k+1} - y_k + 1)$$

Finding the parameter p_k II

Depending on the value of p_k ,

- y_{k+1} is either y_k itself or it is $y_k - 1$

$$p_{k+1} = \left\{ \begin{array}{ll} p_k + 2(x_k + 1) + 1, & \text{if } p_k \text{ is negative} \\ p_k + 2(x_k + 1) + 1 - 2y_k + 2, & \text{if } p_k > 0 \end{array} \right\}$$

Midpoint Circle Algorithm

Midpoint Circle Algorithm

1. Input radius r and circle center (x_c, y_c) , and obtain the first point on the circumference of a circle centered on the origin as

$$(x_0, y_0) = (0, r)$$

2. Calculate the initial value of the decision parameter as

$$p_0 = \frac{5}{4} - r$$

3. At each x_k position, starting at $k = 0$, perform the following test: If $p_k < 0$, the next point along the circle centered on $(0, 0)$ is (x_{k+1}, y_k) and

$$p_{k+1} = p_k + 2x_{k+1} + 1$$

Otherwise, the next point along the circle is $(x_k + 1, y_k - 1)$ and

$$p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_{k+1}$$

where $2x_{k+1} = 2x_k + 2$ and $2y_{k+1} = 2y_k - 2$.

4. Determine symmetry points in the other seven octants.
5. Move each calculated pixel position (x, y) onto the circular path centered on (x_c, y_c) and plot the coordinate values:

$$x = x + x_c \quad y = y + y_c$$

6. Repeat steps 3 through 5 until $x \geq y$.

Tutorial 03

- Find the points to plot for drawing a circle using Midpoint Circle Algorithm for a circle whose centre is $(0,0)$ and radius is 10

Meeting 06

We Saw & Will See

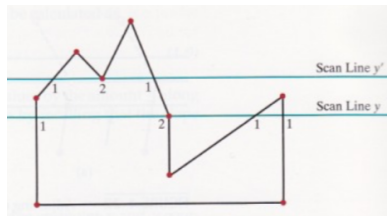
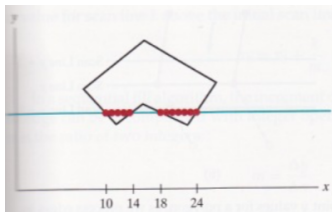
Till Now We Saw...

- Module 1 : Graphic Systems
 - Raster Scan & Random Scan
 - Line Drawing Algorithm : DDA, Bresenham's
 - Bresenham's Circle(Midpoint Circle)

Today We Will See...

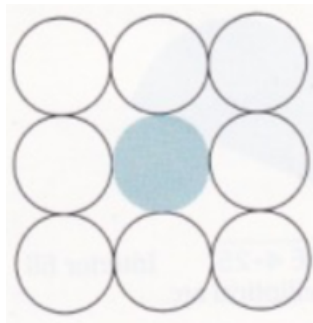
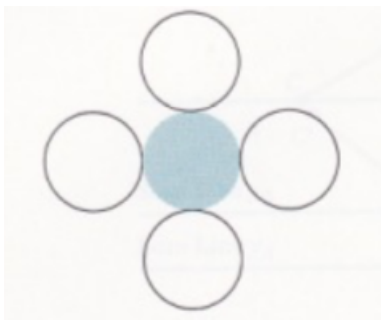
- Filling Algorithms: Scan Line Fill , Flood Fill , Boundary Fill
- Inside-Outside Tests : Odd-Even, Non-zero Winding

Scan Line Filling



- For each scanline, initially flag= 0 and flag= 1 upon seeing a polygon edge
- When flag= 1, fill the pixel with the color to be filled with.
- Toggle flag whenever seeing a polygon edge.
- if seeing a **vertex**, toggle only if the constituent edges are on the different sides of the scanline.

4-connected & 8-connected



- 4-connected : considers 4 pixels as neighbours
- 8-connected : considers 8 pixels as neighbours

Flood Fill

- Fill with 'fillColor' on all contiguous pixels with 'oldColor'.

```
void floodFill4 (int x, int y, int fillColor, int oldColor)
{
    if (getPixel (x, y) == oldColor) {
        setColor (fillColor);
        setPixel (x, y);
        floodFill4 (x+1, y, fillColor, oldColor);
        floodFill4 (x-1, y, fillColor, oldColor);
        floodFill4 (x, y+1, fillColor, oldColor);
        floodFill4 (x, y-1, fillColor, oldColor);
    }
}
```

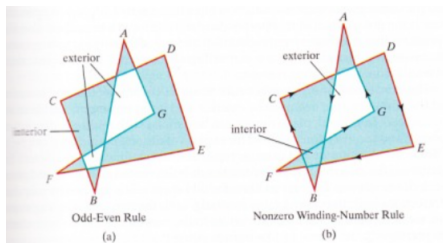
Boundary Fill

- Fill with 'fill' color, on all contiguous pixels till finding pixels with 'boundary' color.

```
void boundaryFill4 (int x, int y, int fill, int boundary)
{
    int current;

    current = getPixel (x, y);
    if ((current != boundary) && (current != fill)) {
        setColor (fill);
        setPixel (x, y);
        boundaryFill4 (x+1, y, fill, boundary);
        boundaryFill4 (x-1, y, fill, boundary);
        boundaryFill4 (x, y+1, fill, boundary);
        boundaryFill4 (x, y-1, fill, boundary);
    }
}
```

Inside-Outside Test



- **Odd-Even Rule** : draw a line from a starting point, not passing through any vertex, to outside the polygon. If odd number of edges are cut, then the starting point is an interior point, otherwise exterior point.
- **Non-Zero Winding Number** : similar to the previous case draw a line. count the number of edges with direction as shown in figure. add 1 for edges which are from left to right and subtract 1 which are from right to left. If we get a non-zero number in the end, the starting point is an interior point, else an exterior point.